Welcome

Digital and Computer Vision

Lesson 3: Session 4

Morphological Operations

Summary of the previous session: Object Detection with Haar Cascades

In the previous session we have covered :

Understanding Haar Cascades for face detection.
Object detection with HOG-SVM algorithm
Digits classification with HOG-SVM algorithm

Summary of the current session: Morphological Operations

In this session we will covered the following:

Morphological Operations
Dilation, Erosion, Opening, and Closing.
Removing noise using morphological transformations.

Exercise from previous session

Study and explain the k-Nearest Neighbors (KNN) algorithm. Then, implement a Python program to train a classifier for handwritten digits using KNN.

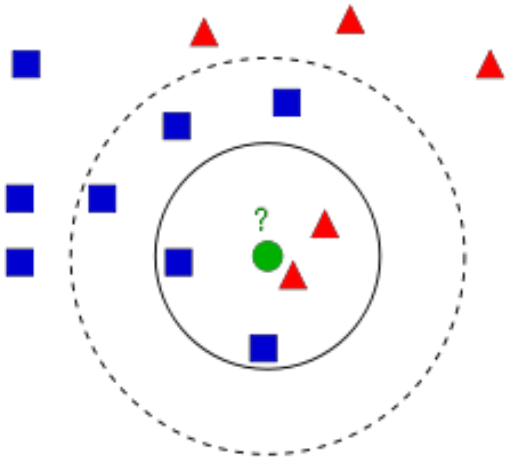
Additionally, compare the performance and characteristics of KNN with the HOG-SVM approach. Discuss their strengths, weaknesses, and suitability for different applications.

Use the web resources provided below for reference.

https://docs.opencv.org/4.x/d5/d26/tutorial_py_knn_understanding.html

k-Nearest Neighbours (KNN) algorithm

k-Nearest Neighbour (k-NN) is a simple and intuitive, machine learning algorithm used for classification. It makes predictions based on the closest match(es) of the test data in the feature space.



https://docs.opencv.org/4.x/d5/d26/tutorial_py_knn_understanding.html

In the image, there are two type of points: Blue Squares and Red Triangles. We refer to each type as a Class. You can consider a feature space as a space where all data are projected. Now consider what happens if a new member comes, which is shown as the green circle. He should be added to one of these Blue or Red classes. We call that process, Classification. One simple method is to check who is his nearest neighbour. From the image, it is clear that it is a member of the Red Triangle class. So he is classified as a Red Triangle.

k-Nearest Neighbours (KNN) algorithm

But there is a problem with this approach! Red Triangle may be the nearest neighbour, but what if there are also a lot of Blue Squares nearby? Then Blue Squares have more strength in that locality than Red Triangles, so just checking the nearest one is not sufficient. Instead we may want to check some k nearest points. Then whichever class is the majority amongst them, the new guy should belong to that class. In our image, let's take k=3, i.e. consider the 3 nearest neighbours. The new member has two Red neighbours and one Blue neighbour (there are two Blues equidistant, but since k=3, we can take only one of them), so again he should be added to Red family. But what if we take k=7? Then he has 5 Blue neighbours and 2 Red neighbours and should be added to the Blue family. The result will vary with the selected value of k.

How KNN works

1. Choose k, the number of neighbours to look at.

2. Compute the distance from the new input point to all points in the training data.

3. Select the k nearest neighbours (smallest distances).

4, Vote: For classification, the most frequent class among those k neighbours is chosen as the prediction.

Common Distance Metrics:
1. Euclidean: straight-line distance (most common)
2. Manhattan: block-wise distance

KNN Pros and Cons

Pros:
Very simple to implement
No training needed (lazy learner)
Adapts to new data easily

Cons:
Slow at prediction time (computes distance to all training points)
Sensitive to noisy data and irrelevant features
Choice of k and distance metric matters

KNN vs HOG-SVM

Aspect	KNN	HOG-SVM
Type	Instance-based (lazy) learning	Feature-based + supervised learning (discriminative model)
Feature Representation	Raw pixel values or simple descriptors	HOG (Histogram of Oriented Gradients) for shape features
Classifier	Distance-based voting	SVM (max-margin classifier)
Training Time	Very fast (no training)	Moderate (training SVM on extracted features)
Prediction Time	Slow (distance calculation for all data)	Fast (especially with linear SVM)
Memory Usage	High (stores all training data)	Low (model stores support vectors only)

KNN training example code 1/2

```
import cv2 as cv
import numpy as np

img = cv.imread('..\Images\digits.png')
gray = cv.cvtColor(img,cv.COLOR_BGR2GRAY)

# Now we split the image to 5000 cells, each 20x20 size
cells = [np.hsplit(row,100) for row in np.vsplit(gray,50)]

# Make it into a Numpy array: its size will be (50,100,20,20)
x = np.array(cells)

# Now we prepare the training data and test data
train = x[:,:50].reshape(-1,400).astype(np.float32) # Size = (2500,400)
test = x[:,50:100].reshape(-1,400).astype(np.float32) # Size = (2500,400)
print("Shape:", test.shape)
print("Dtype:", test.dtype)
```

KNN training example code 2/2

```
# Create labels for train and test data
k = np.arange(10)
train_labels = np.repeat(k,250)[:,np.newaxis]
test_labels = train_labels.copy()

# Initiate kNN, train it on the training data, then test it with the test data with k=1
knn = cv.ml.KNearest_create()
knn.train(train, cv.ml.ROW_SAMPLE, train_labels)
# Save the data
np.savez('knn_data.npz',train=train, train_labels=train_labels)
ret,result,neighbours,dist = knn.findNearest(test,k=5)

# Now we check the accuracy of classification
# For that, compare the result with test_labels and check which are wrong
matches = result==test_labels
correct = np.count_nonzero(matches)
accuracy = correct*100.0/result.size
print( accuracy )
```

KNN detection example code 1/2

```
import cv2 as cv
import numpy as np

img = cv.imread('..\Images\dign_1.png')
gray = cv.cvtColor(img,cv.COLOR_BGR2GRAY)
img_resized = cv.resize(gray, (20, 20))  # Resize to 20x20
# Flatten the image to a 1D vector
sample = img_resized.flatten()
# Convert to a 2D array with 1 sample and 400 features (20x20)
sample = sample.reshape(1, -1).astype(np.float32)

# Load the data
with np.load('knn_data.npz') as data:
    print( data.files )
    train = data['train']
    train_labels = data['train_labels']
```

KNN detection example code 2/2

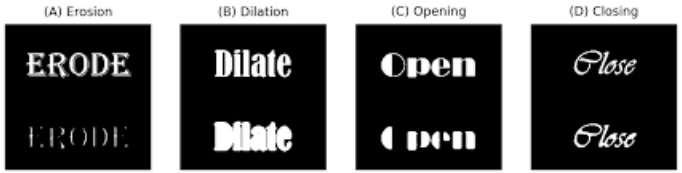
```
# Initiate kNN, train it on the training data, then test it with the test data with k=1
knn = cv.ml.KNearest_create()
knn.train(train, cv.ml.ROW_SAMPLE, train_labels)

ret,result,neighbours,dist = knn.findNearest(sample,k=5)
print( ret,result,neighbours,dist )
```

Morphological operations

Image filters and thresholds enable us to detect structures of various shapes and sizes for different applications. Nevertheless, despite our best efforts, the binary images produced by our thresholds often still contain inaccurate or undesirable detected regions. They could benefit from some extra cleaning up.

Morphological operations like erosion and dilation are fundamental image processing techniques, especially useful in binary images (but applicable to grayscale too). They are primarily used to modify the structure or shape of objects in an image. Two basic morphological operators are Erosion and Dilation. Then its variant forms like Opening, Closing,

Both operations use a kernel also called structuring element, that slides over the image to examine and modify the neighborhood of each pixel.



https://bioimagebook.github.io/chapters/2-processing/5-morph/morph.html

Erosion

Erosion erodes away the boundaries of foreground object (Always try to keep foreground in white). So what it does? The kernel slides through the image (as in 2D convolution). A pixel in the original image (either 1 or 0) will be considered 1 only if all the pixels under the kernel is 1, otherwise it is eroded (made to zero).

NB Different from the process of convolution, the morphological operation is logical (e.g. AND, OR, NOT) rather than arithmetic.

What it does:

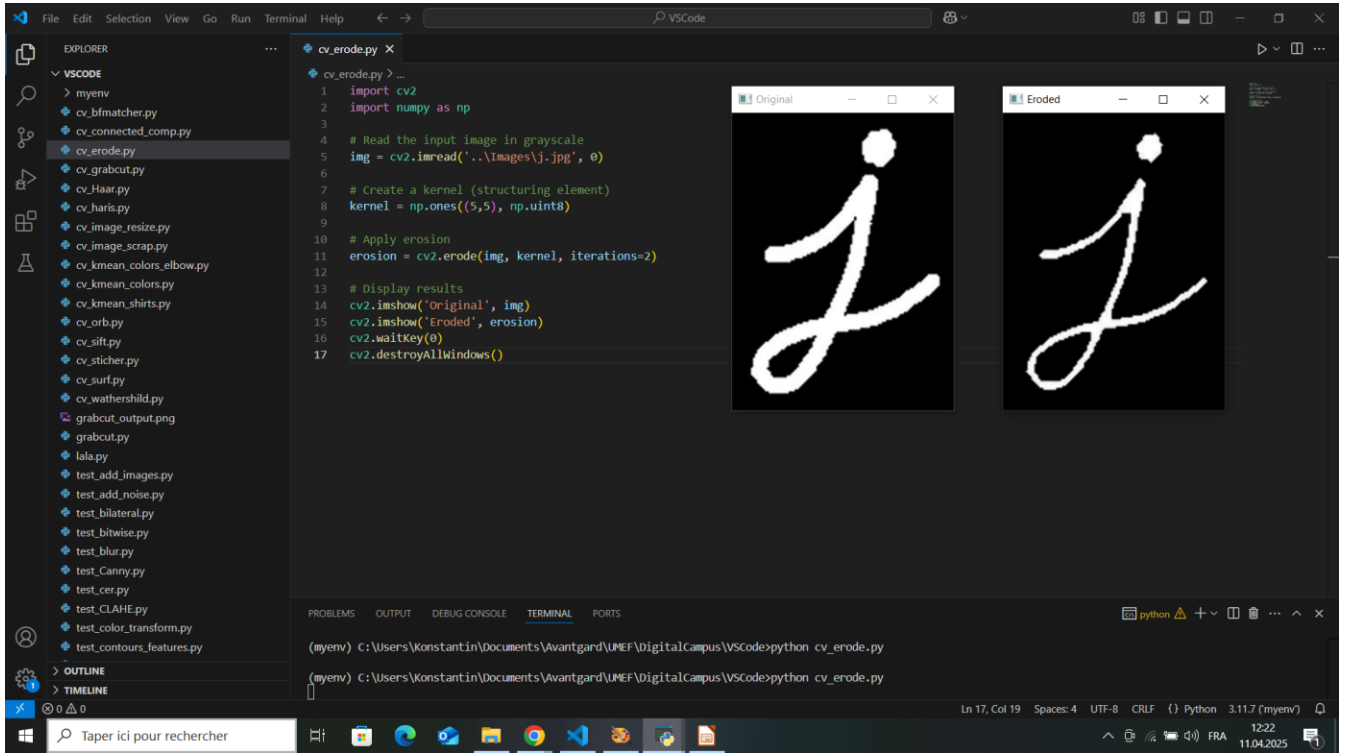
Shrinks white regions (foreground).

Removes noise.

Useful for eliminating small white patches or separating connected objects.

Erosion code example

```
import cv2
import numpy as np

# Read the input image in grayscale
img = cv2.imread('..\Images\j.jpg', 0)

# Create a kernel (structuring element)
kernel = np.ones((5,5), np.uint8)

# Apply erosion
erosion = cv2.erode(img, kernel, iterations=2)

# Display results
cv2.imshow('Original', img)
cv2.imshow('Eroded', erosion)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Dilation

Dilation is just opposite of erosion. It increases the white region in the image or size of foreground object increases.
What it does:

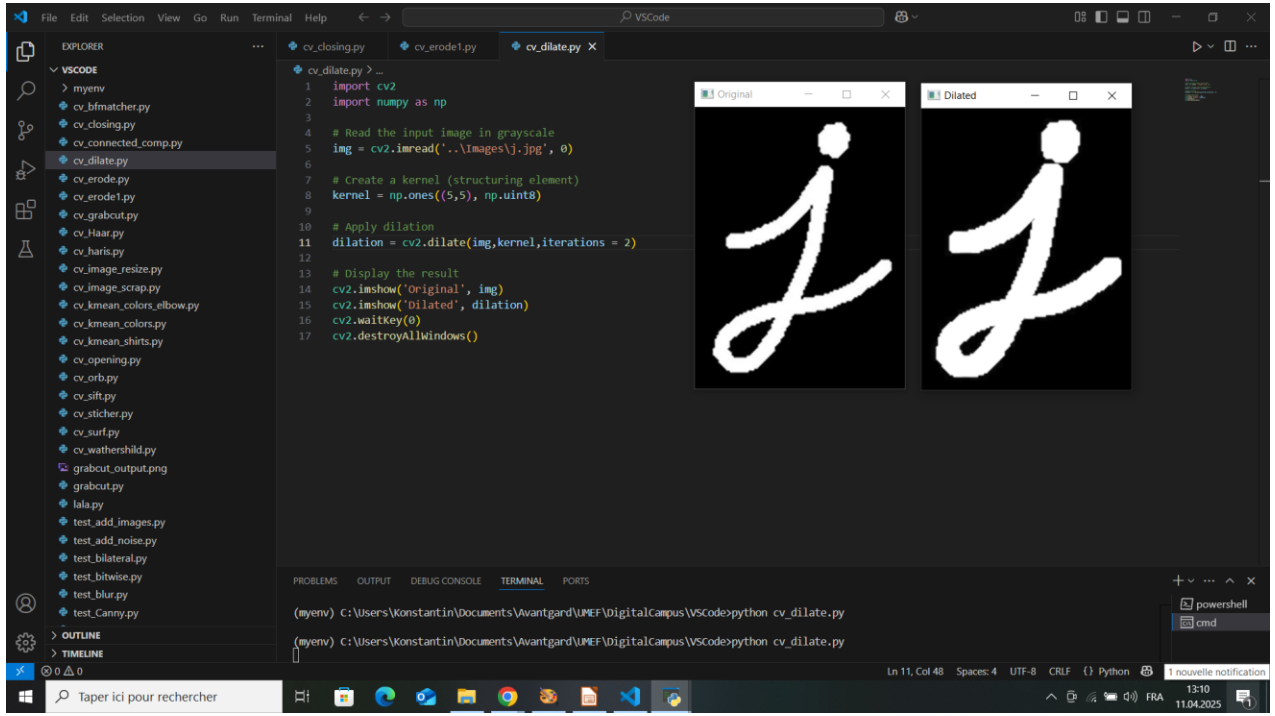
Expands white regions.
Fills small holes.

How:
If any part of the structuring element hits a white pixel, the whole neighborhood becomes white.

Normally, in cases like noise removal, erosion is followed by dilation. Because, erosion removes white noises, but it also shrinks our object. So we dilate it. Since noise is gone, they won't come back, but our object area increases. It is also useful in joining broken parts of an object.

Dilation – code example

```
import cv2
import numpy as np

# Read the input image in grayscale
img = cv2.imread('sample_image.png', 0)

# Create a kernel (structuring element)
kernel = np.ones((5,5), np.uint8)

# Apply dilation
dilation = cv2.dilate(img,kernel,iterations = 1)

# Display the result
cv2.imshow('Original', img)
cv2.imshow('Dilated', dilation)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Opening

Opening is a morphological operation in image processing that consists of Erosion followed by Dilation

Purpose of Opening :

Removes small white noise (tiny white dots or blobs in black background)
Separates objects that are slightly connected

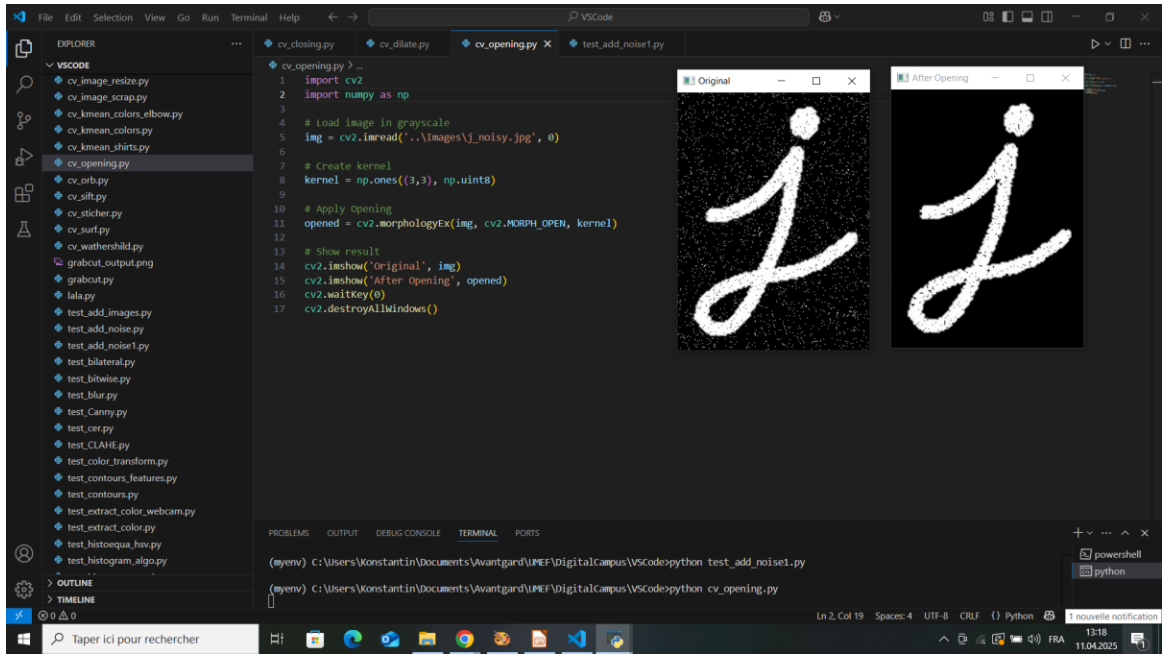
Keeps the overall shape and size of larger foreground objects mostly intact

How It Works :

Erosion removes white pixels (shrinks objects), getting rid of small noise.
Dilation grows the remaining white areas back to their original size.

Opening – code example

```
import cv2
import numpy as np

# Load image in grayscale
img = cv2.imread('..\Images\j_noisy.jpg', 0)

# Create kernel
kernel = np.ones((3,3), np.uint8)

# Apply Opening
opened = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)

# Show result
cv2.imshow('Original', img)
cv2.imshow('After Opening', opened)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Closing

Closing is a morphological operation that is the reverse of Opening - Dilation followed by Erosion

Purpose of Closing :

Fills small black holes or gaps inside white (foreground) objects

Connects nearby white regions (useful when an object is broken up slightly)

Maintains the overall shape and size of foreground objects

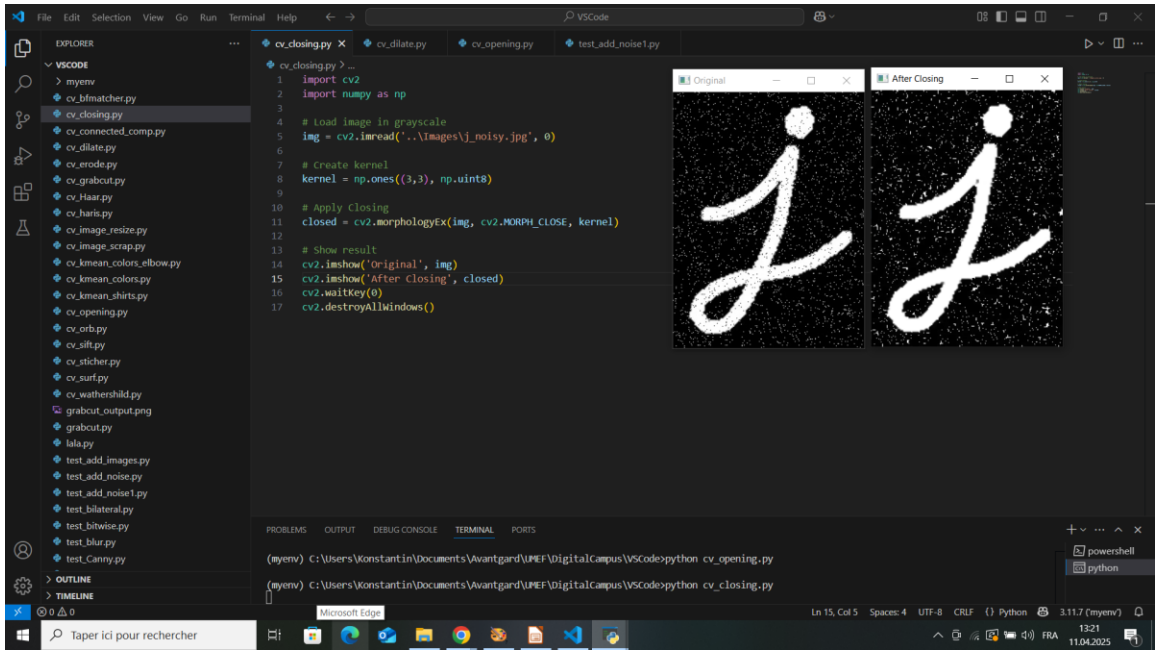
How It Works :

Dilation expands white regions, bridging small gaps or holes.

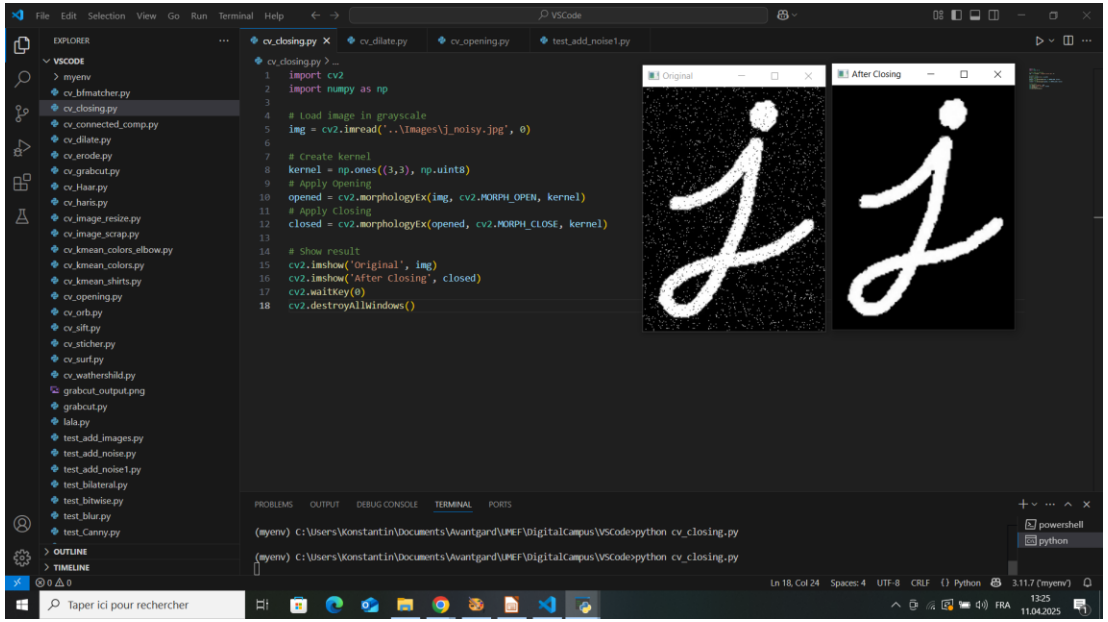
Erosion trims the expanded area back, restoring the object's size but keeping the gaps closed.

Closing code example

```
import cv2
import numpy as np

# Load image in grayscale
img = cv2.imread('..\Images\j_noisy.jpg', 0)

# Create kernel
kernel = np.ones((3,3), np.uint8)

# Apply Closing
closed = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)

# Show result
cv2.imshow('Original', img)
cv2.imshow('After Closing', closed)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Remouving the noise code example




```
import cv2
import numpy as np

# Load image in grayscale
img = cv2.imread('..\Images\j_noisy.jpg', 0)

# Create kernel
kernel = np.ones((3,3), np.uint8)
# Apply Opening
opened = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)
# Apply Closing
closed = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, kernel)

# Show result
cv2.imshow('Original', img)
cv2.imshow('After Closing', closed)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Morphological Gradient

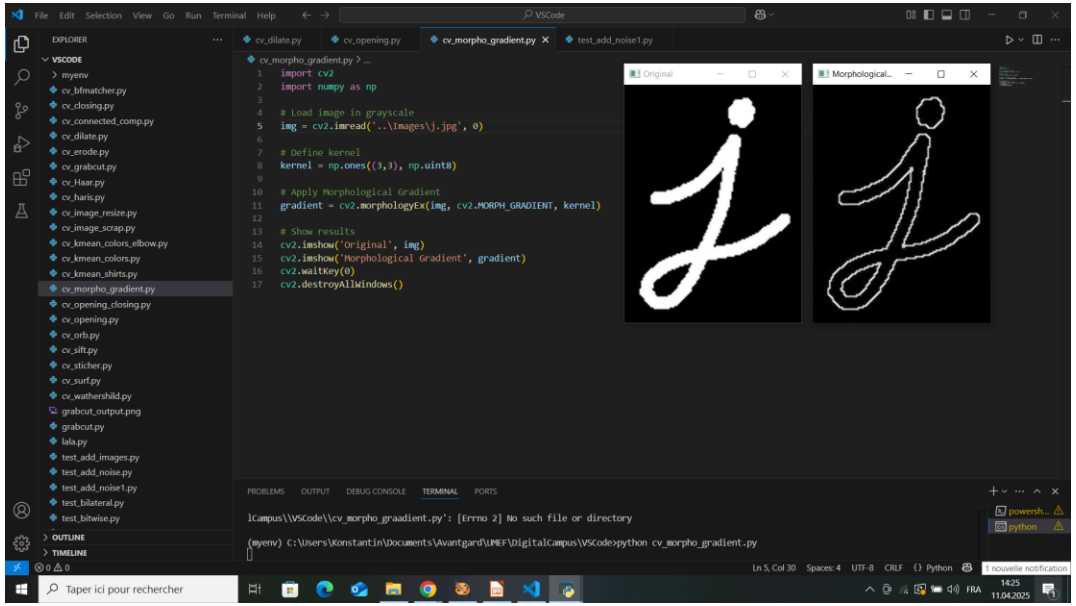
The Morphological Gradient is a morphological image processing operation that highlights the edges or boundaries of objects in a binary or grayscale image. It computes the difference between the dilated and eroded versions of the image.

What does it do?

Emphasizes contours or outlines of objects.
Removes solid areas, keeping only edge information.
Works well on binary images and can be applied to grayscale images too.

Morphological Gradient - code example

```
import cv2
import numpy as np

# Load image in grayscale
img = cv2.imread('..\Images\j.jpg', 0)

# Define kernel
kernel = np.ones((3,3), np.uint8)

# Apply Morphological Gradient
gradient = cv2.morphologyEx(img, cv2.MORPH_GRADIENT, kernel)

# Show results
cv2.imshow('Original', img)
cv2.imshow('Morphological Gradient', gradient)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Morphological operations and ML

Morphological operations are image processing techniques but play a key preprocessing role in many computer vision learning algorithms.

Why Is Preprocessing Important for ML?

ML models (especially traditional ones like SVM, KNN) rely on clean and consistent inputs.

Morphological preprocessing boosts the signal, removes noise, and can significantly improve model accuracy.

For example:

Before passing a digit to a trained SVM model (like your digit classifier), you might erode the image to remove tiny specks or dilate to reinforce the strokes.

In segmentation (e.g., cell counting), after thresholding, you might dilate then erode (closing) to ensure proper boundaries for ML-based counting.

Morphological operations and Deep Learning

Even in deep learning, morphological ops can help:

Preprocessing masks for segmentation networks.

Postprocessing CNN outputs to refine object boundaries.

Though CNNs often learn morphological-like filters, these operations still assist in pipeline cleaning, especially in hybrid systems.

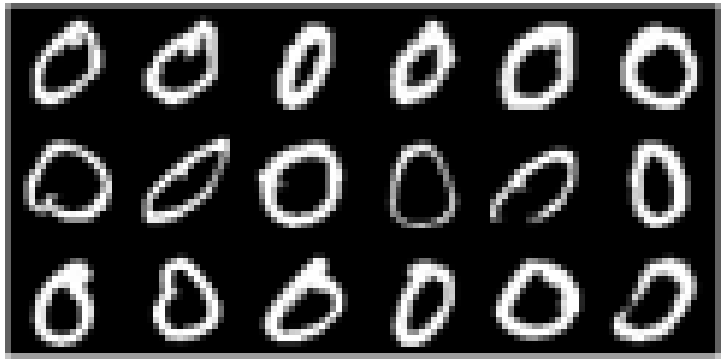
Summary of Morphological operations in ML

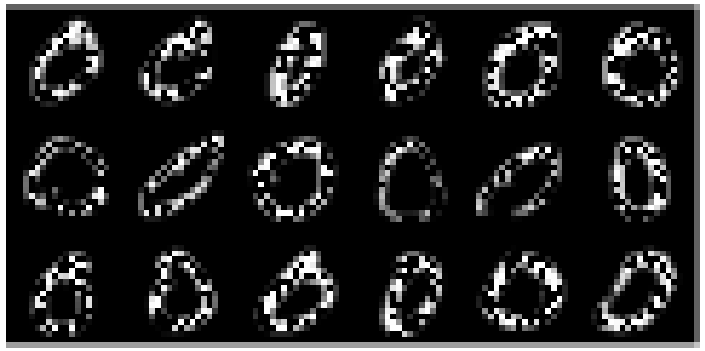
🔍 Task	🤖 Use of Morphological Ops	📘 Learning Algorithm
Digit/character recognition	Clean up and isolate digits before feature extraction	KNN, SVM, HOG-SVM
Object detection	Separate overlapping objects, refine contours	Haar cascades, HOG-SVM
Feature extraction	Highlight relevant structures and remove noise	Any ML-based CV task
Image segmentation	Post-process segmentation masks (remove small blobs/fill holes)	GrabCut, Watershed
Contour analysis	Ensure smooth, well-defined contours for shape features	Shape descriptors

Morphological cleaning for HOG-SVM digits decoder

Lets randomly sets some pixels to 0 (black) in the image digits.png which we use to train our HOG-SVM handwritten digit classifier. The code is given below :

```
import cv2
import numpy as np

img = cv2.imread('..\Images\digits.png', cv2.IMREAD_GRAYSCALE)
output = img.copy()
# Define a threshold mask to only drop pixels on light regions (digits)
digit_mask = img > 128  # adjust this if digits are lighter
# Create a dropout mask (e.g., 50% chance to drop a pixel)
dropout_mask = np.random.rand(*img.shape) < 0.5
# Combine both masks: drop only pixels in digit areas
final_mask = digit_mask & dropout_mask
# Apply dropout: set selected pixels to 0
output[final_mask] = 0
# Save or show result
cv2.imwrite('..\Images\digits_with_gaps.png', output)
```

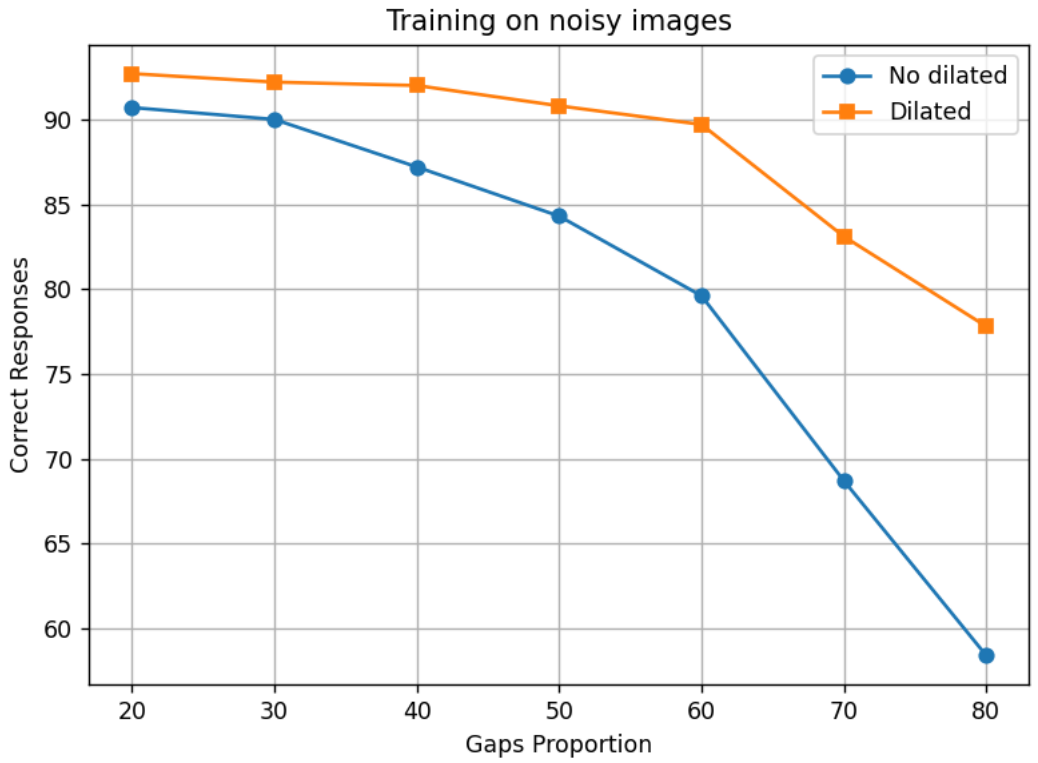




Morphological cleaning for HOG-SVM training code example

```
……...
img = cv.imread('..\Images\digits_with_gaps.png')
if img is None:
    raise Exception("we need the digits.png image from samples/data here !")

# Morphological operation: Opening (removes noise)
kernel = np.ones((2, 2), np.uint8)
# Apply dilation
img = cv.dilate(img,kernel,iterations = 1)
……
```

Whithout dilation I get 70.92 % correct results during testing
After dilation I get 84.48 % correct results during testing



Morphological Text Line Segmentation in Scanned Documents

When you have a scanned document with printed or handwritten text, you might want to segment the text lines for further processing, such as OCR (Optical Character Recognition). Morphological operations are very effective here.

What We'll Do:

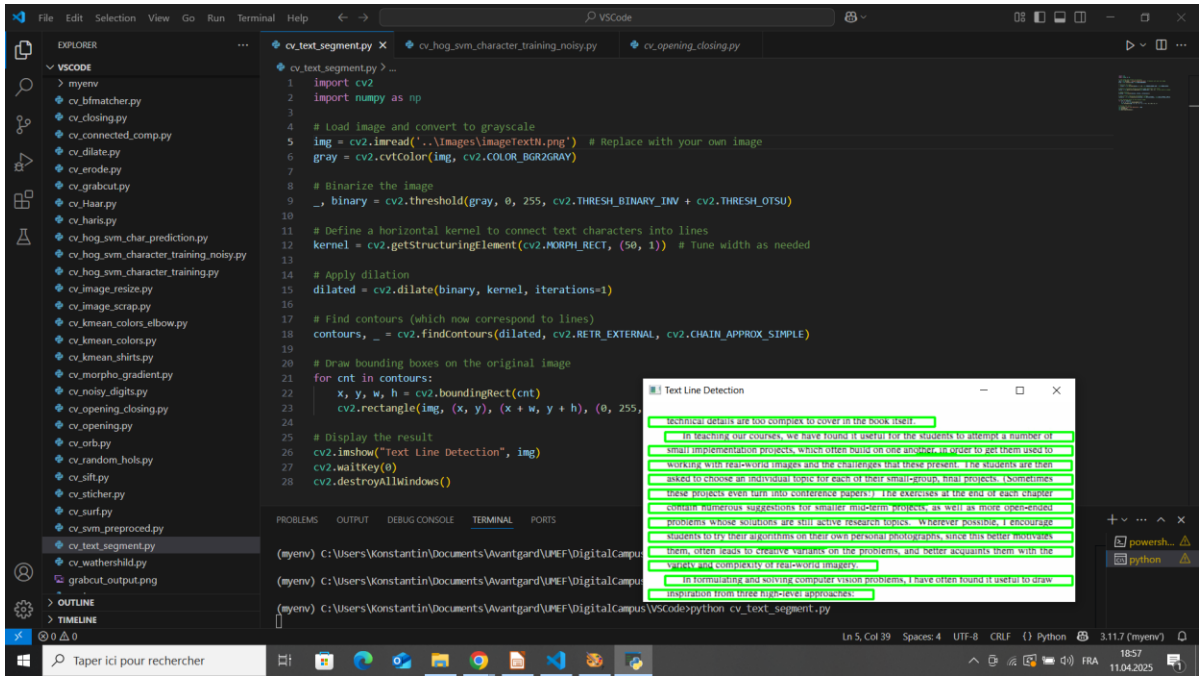
Load a scanned text image.
Convert to grayscale and threshold it to binary.
Apply morphological dilation with a horizontal kernel to connect characters into lines.
Use contour detection to extract bounding boxes of text lines.

Morphological Text Line Segmentation code example 1/2

```
import cv2
import numpy as np

# Load image and convert to grayscale
img = cv2.imread('..\Images\imageTextR.png')  # Replace with your own image
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Binarize the image
_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

# Define a horizontal custom kernel to connect text characters into lines
#The kernel shape and size determines how the connection happens
# In this case, a wide horizontal kernel targets lines of text.

kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (50, 1))  # Tune width as needed

# Apply dilation
dilated = cv2.dilate(binary, kernel, iterations=1)

# Find contours (which now correspond to lines)
```

Morphological Text Line Segmentation code example 2/2

```
# Apply dilation
dilated = cv2.dilate(binary, kernel, iterations=1)

# Find contours (which now correspond to lines)
contours, _ = cv2.findContours(dilated, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# Draw bounding boxes on the original image
for cnt in contours:
    x, y, w, h = cv2.boundingRect(cnt)
    cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)

# Display the result
cv2.imshow("Text Line Detection", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Recapitulation of Image Processing and Computer Vision Algorithms 1/6

🔄 Geometric Transformations

Method	Description	Use / Task
Rotate / Scale / Translate	Basic transformations that change position, size, or orientation of the image.	Image alignment, preprocessing
Perspective Transform / Warp	Alters perspective to view image from different angles.	Document scanning, object flattening

🎨 Color and Format Conversion

Method	Description	Use / Task
Grayscale, HSV, RGB	Converts images between color spaces.	Preprocessing for segmentation, detection

Recapitulation of Image Processing and Computer Vision Algorithms 2/6

➕ Image Arithmetic

Method	Description	Use / Task
Add / Subtract / Blend	Combines images pixel-wise or subtracts them.	Motion detection, overlays, visual effects

🌫 Filtering and Smoothing

Method	Description	Use / Task
Gaussian Blur	Smooths image by averaging with a Gaussian kernel.	Noise reduction, pre-edge detection
Median Filter	Replaces pixel value with median of neighborhood.	Removes salt-and-pepper noise
Bilateral Filter	Preserves edges while smoothing similar pixel regions.	Facial smoothing, edge-preserving blur

Recapitulation of Image Processing and Computer Vision Algorithms 3/6

📊 Histogram Techniques

Method	Description	Use / Task
Histogram Equalization	Enhances contrast by spreading out intensity distribution.	Improves visibility in low-contrast images

🧱 Thresholding & Binarization

Method	Description	Use / Task
Simple Threshold	Applies fixed cutoff to binarize pixels.	OCR, basic object segmentation
Adaptive Threshold	Varies threshold over regions.	Uneven lighting conditions
Otsu's Method	Automatically selects optimal global threshold.	Global binarization

Recapitulation of Image Processing and Computer Vision Algorithms 4/6

⚡ Edge Detection

Method	Description	Use / Task
Sobel	Detects edges using gradients in X or Y direction.	Edge detection, shape analysis
Laplacian	Second-order derivative filter.	Highlights areas of rapid intensity change
Canny	Multi-stage edge detector with noise reduction.	Robust edge detection for object contours

⬛ Contours and Shapes

Method	Description	Use / Task
Find / Draw Contours	Detects object boundaries.	Shape recognition, object detection
Convex Hull	Smallest convex shape enclosing contour.	Gesture recognition, bounding objects
Shape Approximation	Simplifies contours.	Shape classification, compression

Recapitulation of Image Processing and Computer Vision Algorithms 5/6

⚙️ Morphological Operations

Method	Description	Use / Task
Erosion / Dilation	Shrinks or expands object regions.	Noise removal, gap filling
Opening / Closing	Erosion followed by dilation (Opening) and vice versa (Closing).	Noise cleaning, hole filling
Morphological Gradient	Difference between dilation and erosion.	Highlights edges of objects

🔍 Segmentation

Method	Description	Use / Task
Watershed Algorithm	Segments overlapping objects using contours as barriers.	Object separation in cluttered scenes
K-means Clustering	Partitions image into K regions based on color similarity.	Color-based segmentation
GrabCut	Interactive segmentation using graph cuts.	Foreground extraction

Recapitulation of Image Processing and Computer Vision Algorithms 6/6

🔑 Feature Detection & Matching

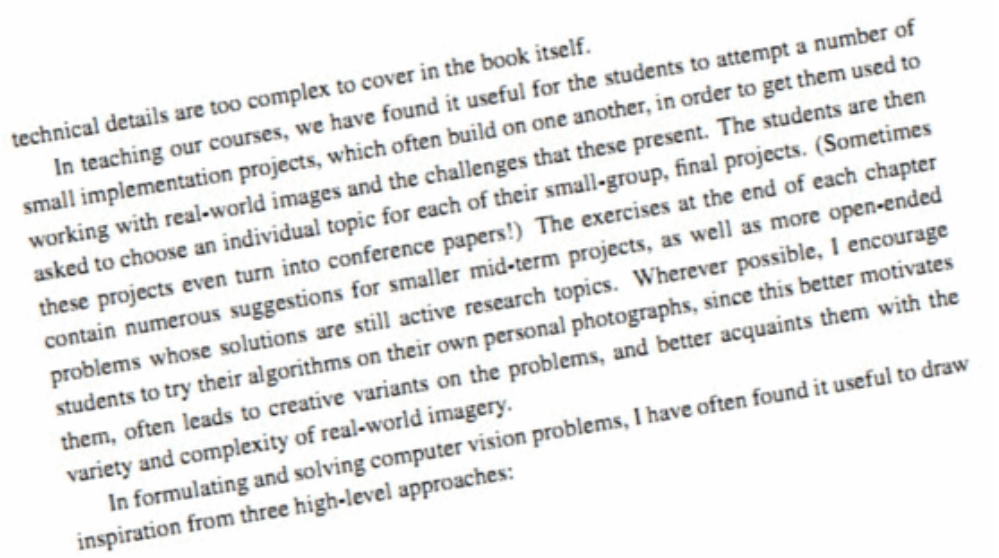
Method	Description	Use / Task
Harris Corner	Detects corners in an image.	Motion tracking, image registration
ORB, SIFT, SURF	Detects and matches features between images.	Object tracking, stitching, recognition

🧠 Object Detection & ML

Method	Description	Use / Task
Haar Cascades	Pre-trained classifiers for face and object detection.	Face detection, real-time apps
HOG + SVM	Uses Histogram of Oriented Gradients and SVM classifier.	Pedestrian/object detection
KNN	Classifies based on nearest training samples.	Digit/character recognition

Exercise

Propose a code for line detection with strongly skewed text like in the image: imageTextR.png



technical details are too complex to cover in the book itself.

In teaching our courses, we have found it useful for the students to attempt a number of small implementation projects, which often build on one another, in order to get them used to working with real-world images and the challenges that these present. The students are then asked to choose an individual topic for each of their small-group, final projects. (Sometimes these projects even turn into conference papers!) The exercises at the end of each chapter contain numerous suggestions for smaller mid-term projects, as well as more open-ended problems whose solutions are still active research topics. Wherever possible, I encourage students to try their algorithms on their own personal photographs, since this better motivates them, often leads to creative variants on the problems, and better acquaints them with the variety and complexity of real-world imagery.

In formulating and solving computer vision problems, I have often found it useful to draw inspiration from three high-level approaches:

In this session we have covered:

Morphological Operations
Dilation, Erosion, Opening, and Closing.
Removing noise using morphological transformations.

In the next session we will covered:

Introduction to Deep Learning & CNNs for Image Processing
Overview of Convolutional Neural Networks (CNNs).
Using TensorFlow/Keras with OpenCV.
Exercise: Building a simple CNN for image classification

THANKS !!!